

Graphics Card Market Analysis - Web Scraping Project

Project Overview

In this project, you will create a web scraper to collect and analyze graphics card data from Newegg.com's GPU section. You'll use Python, BeautifulSoup, and other relevant libraries to extract key information about graphics cards currently available in the market.

Learning Objectives

- Implement web scraping techniques using Python and BeautifulSoup
- Handle pagination in web scraping
- Process and structure collected data
- Practice working with real-world data
- Learn to respect website scraping policies and implement delays

Part 1: Web scrapping

Required Data to Extract

For each graphics card listing, you must collect:

1. Product Image URL
2. Product Title (e.g., "MSI Ventus GeForce RTX 3060 12GB")
3. Product Details:
 - Manufacturer
 - Model Number
 - Rating (number of stars)
4. Pricing Information:
 - Current Price
 - Original Price (if available)
 - Shipping Information

 Screenshot

Project Requirements

Technical Requirements

1. Use Python 3.x
2. Required libraries:
 - requests
 - beautifulsoup4
 - pandas
 - time (for implementing delays)

Functional Requirements

1. Implement proper error handling
2. Include appropriate delays between requests (minimum 2 seconds)
3. Save the scraped data to a CSV file with appropriate column headers
4. Document your code with comments
5. Create a summary of the collected data (e.g., average price, price range)

Part 2: Data Analysis

Required Analysis

1. Price Analysis
 - Calculate average price by brand
 - Identify price ranges and market segments
 - Analyze price-to-memory ratio
 - Compare prices across different GPU series (RTX 3000 vs 4000)
 - Create price distribution visualizations
2. Brand Analysis
 - Market share by brand
 - Average ratings by brand
 - Price comparison across brands
 - Create brand comparison visualizations
3. Memory Configuration Analysis
 - Distribution of memory sizes
 - Price correlation with memory size
 - Popular memory configurations by brand
4. Market Segmentation
 - Entry-level vs Mid-range vs High-end segments
 - Price brackets and their features

Deliverables

1. Python script containing the web scraper
2. CSV file with the collected data
3. README file explaining:
 - Project Description
 - Features
4. Short report analyzing the collected data

Important Notes

- Always check and respect the website's robots.txt file
- Implement appropriate delays between requests to avoid overwhelming the server
- Handle your requests responsibly and ethically
- Be prepared to handle cases where data might be missing or in an unexpected format

```
In [2]: from selenium import webdriver
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import pandas as pd
import time
from selenium.webdriver.chrome.options import Options
from tqdm import tqdm
import requests
```

```
In [3]: url = "https://www.newegg.com/Desktop-Graphics-Cards/SubCategory/ID-48"
PAGES_TO_SCRAPE = 20
service = Service(ChromeDriverManager().install())
options = Options()
options.add_argument("--headless=new")
driver = webdriver.Chrome(options=options)
driver.get(url)
time.sleep(5)

gpu_list = []
page = 1

while page <= PAGES_TO_SCRAPE:
    driver.get(url + '/Page-' + str(page))

    products = container = driver.find_elements(By.CLASS_NAME, 'item-cell')

    for product in tqdm(products, 'in progress'):

        manu = product.find_elements(By.CLASS_NAME, 'item-brand')
        if manu:
            manu = manu[0].find_element(By.TAG_NAME, 'img').get_attribute('title')
        else:
            manu = None
```

```
ratings = product.find_elements(By.CLASS_NAME, 'item-rating')
if ratings:
    ratings = ratings[0].get_attribute('title').split('+ ', 1)[-1]
else:
    ratings = None

reviews = product.find_elements(By.CLASS_NAME, 'item-rating-num')
if reviews:
    reviews = reviews[0].text
else:
    reviews = None

img = product.find_elements(By.TAG_NAME, 'img')
if img:
    img_url = img[0].get_attribute('src')
else:
    img_url = None

ptil = product.find_elements(By.CLASS_NAME, 'item-title')
if ptil:
    ptil = ptil[0].text
else:
    ptil = None

model = product.find_elements(By.CLASS_NAME, 'item-features')
if model:
    model = model[0].text.split(':', 1)[-1]
else:
    model = None

price = product.find_elements(By.CLASS_NAME, 'price-current')
if price:
    price = price[0].text
else:
    price = None

ship = product.find_elements(By.CLASS_NAME, 'price-ship')
if ship:
    ship = ship[0].text
else:
    ship = None

was = product.find_elements(By.CLASS_NAME, 'price-was')
if was:
    was = was[0].text
else:
    was = None

gpu_data = {
    'image' : img_url,
    'manufacturer' : manu,
    'ratings' : ratings,
    'reviews' : reviews,
    'product' : ptil,
    'model' : model,
    'price' : price,
```

